

Give It a Voice

**Ask your own data anything.
Without letting it write SQL.**

Natural language questions answered from your live job market data — built so the model never touches your database. It picks from a menu of questions you wrote, and every answer arrives with the rows it came from.

You have the data. You can't ask it anything.

Real listings, deterministic scores, a ranked gap report, a Verifier that refuses to publish bad output, and a dashboard. To get an answer out of it, you look at a table.

THE QUESTIONS YOUR DASHBOARD CAN'T ANSWER

“Which companies are hiring this week?” “What three skills would unlock the most listings?” “Have things improved this month?” **All answerable. None of them a panel.**

And you can't add a panel for every question you might ever think of.

The obvious build is the dangerous one.

TEXT-TO-SQL

~15 lines. taught in a lot of tutorials. don't.

```
sql = gemini(f"schema: {SCHEMA}\nquestion: {q}\nwrite SQL:")
```

```
rows = db.execute(sql)      # ← arbitrary generated SQL
```

It's SELECT today. Nothing in that design prevents a DROP TABLE — you're relying on the model choosing not to. And some of the text in your prompt came off the internet, out of job descriptions you scraped.

That's the shape of a prompt injection — and it doesn't require anyone to be **targeting you**.

What you built today

- 1 Seven **parameterised query tools** you wrote — clamped params, tests.
- 2 A **router** that picks one and never composes a query.
- 3 A **phrasing call** that may only use numbers it was handed.
- 4 Answers that ship with **their rows**.
- 5 A **refusal** that names what can be asked instead.

SQL generated by a language model: **none**. The model appears twice and touches the database zero times.

Your data, answerable

The build

- 1 5+ tools working (7 if you use them all)
- 2 No model-generated SQL **anywhere**
- 3 Params validated and clamped
- 4 Rows shown with every answer

And one thing that's yours

- 1 Add **one tool I didn't write**
- 2 A question about **your** career
- 3 Should take ten minutes
- 4 If it takes longer, your registry isn't clean

Film the refusal.

Ask it something it genuinely can't answer. **Fourth week running — you'll have guessed.**

RULE 45

> should I take a pay cut for a remote role?

I can't answer that from your data. Here's what I can:

- which companies are hiring, and when
- your best-matching listings, and why
- your top skill gaps, and what they block

A system that always produces **something** trains you to verify every answer — which costs more than the feature saves.

AN HONEST NOTE BEFORE YOU POST ABOUT IT

This is not a chatbot.

No memory, no follow-ups, no conversation. It answers a fixed set of questions in natural language.

AND THAT'S THE DESIGN, NOT A SHORTCUT

Seven questions answered reliably, with receipts, refusing the rest, is worth more than **anything answered and nothing trusted.**

Asked if you built a chatbot? You built a **query interface with a language model at the edges** — the better answer to anyone who knows the difference.

Thursday it goes public.

Hugging Face Spaces, a real URL, running on a schedule with GitHub Actions — your agent wakes up every morning without you and updates a dashboard anyone can open. Last session, and it earns the badge.

But today your system started answering questions instead of just displaying data — without ever letting a model near your database.

Film the refusal. Add a tool. See you Thursday.